Travel Booking Management System

Requirements & Design Documentation

Oct 24, 2026

1. Introduction

This document specifies the requirements and design of the Travel Booking Management System, a cross-platform "Manage My Booking" application for a travel agency. It covers functional and non-functional requirements, UML diagrams (use case, class, sequence, activity), the database design (ER diagram and schema), system architecture, security measures, and the testing approach, as required by the project specification's Software Engineering & Security Requirements section.

1.1 Project overview

When a customer books a holiday, flight, hotel, or transfer through the agency's website, the booking becomes available inside this system. Customers can view, manage, and track the complete booking lifecycle — payments, travel documents, passenger details, and support tickets — from a mobile, desktop, or web application. Agency staff use the same system through an administrative portal to manage bookings, record payments, respond to support tickets, and maintain customer records.

1.2 Scope

The system consists of two parts, both implemented and tested:

- **Backend API** (Node.js, Express, PostgreSQL via Sequelize): authentication, booking lifecycle management, payments, documents, support tickets, notifications, and staff administration. 62 automated end-to-end tests cover every endpoint.
- **Client application** (Flutter, targeting Windows, Linux, Android, iOS, and web from one codebase): 23 screens across a customer-facing app and a staff administrative portal, covering every feature in the Customer Features and Agency/Admin System sections of the specification. 43 automated unit tests cover the client's data models and business logic.

1.3 Actors

Actor	Description
Customer	A traveller with a booking. Registers, logs in, and manages their own bookings, payments, documents, passengers, and support tickets.
Guest	An unregistered visitor who looks up a booking using its reference and email address, with read-only access.
Booking Agent	Staff who create and edit bookings, update status, assign bookings, and manage flight/hotel/transfer details and documents.
Accounts Staff	Staff who record payments, issue refunds, and manage documents.
Support Agent	Staff who manage support tickets: reply, change status, and reassign.
Administrator	Staff with every permission above, plus staff account management.

Role permissions are enforced on both the server (route guards reject a request before it runs) and the client (the interface hides actions a role cannot perform).

2. Functional Requirements

2.1 Customer

ID	Requirement
FR-1	Register, log in, log out, and reset a forgotten password
FR-2	View and edit profile details (name, phone)
FR-3	View upcoming, active, and previous bookings, grouped automatically by status and date

ID	Requirement
FR-4	Look up a booking by reference and email without logging in (guest access, read-only)
FR-5	View full booking details: booking info, destination, flight, hotel, and transfer
FR-6	View the booking's progress through its lifecycle (Booking Created → Payment Received → Hotel Confirmed → Flights Confirmed → Documents Issued → Ready to Travel → Completed)
FR-7	View payment summary (total price, amount paid, remaining balance, due date) and full payment history with receipts
FR-8	View and download travel documents (confirmation, tickets, vouchers, insurance, visa, ATOL certificate, invoice, receipts)
FR-9	Add, view, edit, and delete passengers on a booking, up to the number of travellers booked
FR-10	View a passenger's protected details (passport number, expiry) only on explicit request, masked by default
FR-11	Raise a support ticket against a booking or as general support, and reply in its thread
FR-12	Receive and view in-app notifications for status changes, payments, and documents
FR-13	See a travel countdown for the next upcoming trip

2.2 Agency Staff

ID	Requirement
FR-14	Sign in with a separate staff account, independent of any customer session on the same device

ID	Requirement
FR-15	View agency-wide dashboard metrics: total, pending, confirmed, and cancelled bookings; customers travelling soon; outstanding payments; open tickets
FR-16	Search and view customer records, including their bookings, tickets, and contact history
FR-17	Log a contact note against a customer (phone, email, in-app, in-person)
FR-18	Create a booking manually and edit destination, dates, traveller counts, and notes (Administrator, Booking Agent)
FR-19	Change a booking's status and assign it to a staff member (Administrator, Booking Agent)
FR-20	Create and edit flight, hotel, and transfer details for a booking (Administrator, Booking Agent)
FR-21	Record a payment and issue a refund against a booking (Administrator, Accounts Staff)
FR-22	Upload and delete travel documents (Administrator, Booking Agent, Accounts Staff)
FR-23	Reply to, change the status of, and reassign support tickets (every role can reply; Administrator and Support Agent can change status and assignment)
FR-24	Create staff accounts, and edit a staff member's name, role, and active status (Administrator only)

2.3 System (website integration)

ID	Requirement
FR-25	Accept a new booking pushed from the agency website via an authenticated API call, creating a customer account automatically if one does not already exist for that email
FR-26	Notify the customer automatically when a booking's status changes or a document is uploaded

3. Non-Functional Requirements

Category	Requirement	How it is met
Security	Passwords are never stored in plain text	bcrypt hashing (10 salt rounds) for both customer and staff accounts
Security	Sensitive personal data is encrypted at rest	Passport numbers encrypted with AES-256-GCM before storage; decrypted transparently on read
Security	Sessions expire automatically	JWT tokens: 7 days for customers, 12 hours for staff; a deactivated staff account is rejected immediately, not just at next login
Security	The system resists brute-force and enumeration attacks	Rate limiting: 100 requests / 15 min on auth endpoints, 20 / 15 min on public booking lookup
Security	Inputs are validated before reaching the database	Server-side validation on every write endpoint (dates, counts, prices, emails); field whitelisting prevents mass-assignment of protected fields such as status or customerId

Category	Requirement	How it is met
Security	Machine-to-machine integration is authenticated	The website-to-system booking sync endpoint requires a shared API key, compared using a timing-safe check
Security	File access is protected and time-limited	Document downloads use a 90-second signed link scoped to one document; it cannot be reused as a login credential
Reliability	The system degrades gracefully under failure	Centralised error handling returns consistent JSON errors; internal error detail is never leaked to the client
Usability	The interface adapts to the device it runs on	Flutter client runs unmodified on phone, tablet, desktop, and web; the staff portal switches between a navigation drawer (phone) and a side rail (wide screens)
Usability	Protected data is hidden by default	Passport numbers are masked in the UI until the customer explicitly chooses to reveal them
Performance	Common lookups respond quickly under normal load	Indexed primary/foreign keys; booking lists avoid N+1 queries via eager loading of associations
Maintainability	The codebase is verifiably correct	62 backend end-to-end tests and 43 Flutter unit tests, run on every change; documented REST API reference (API.md)
Portability	One codebase serves every required platform	Flutter targets Windows, Linux, Android, iOS, and web from a single source tree, satisfying the specification's platform requirement

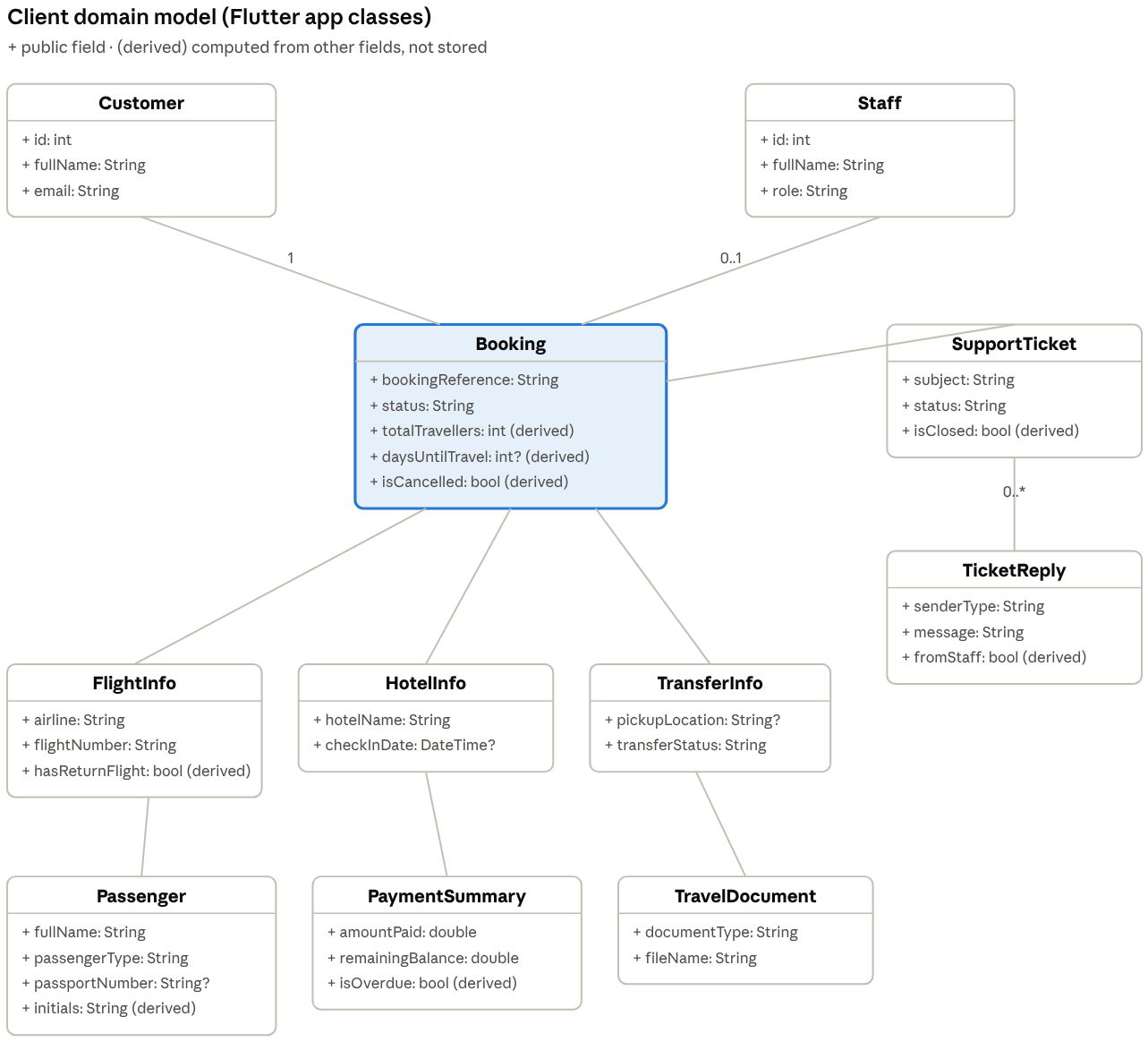
Category	Requirement	How it is met
Compliance	Regulatory certificates are tracked per booking	ATOL Certificate is a first-class document type, consistent with UK travel agency requirements

4. User Requirements vs System Requirements

User requirements state what a traveller or staff member needs in plain language. System requirements state precisely what the software does to satisfy that need, and are directly traceable to an API endpoint.

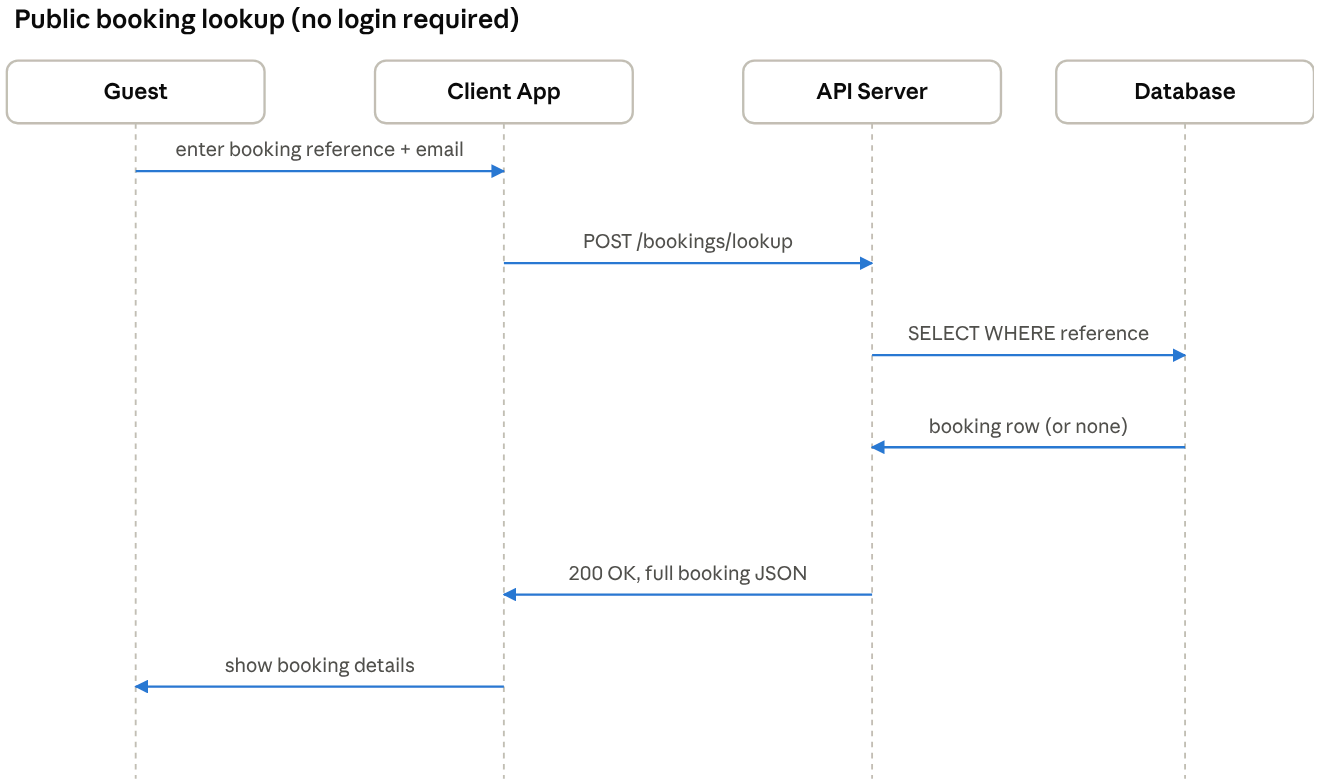
User requirement	System requirement
"I want to find my booking without creating an account."	`POST /api/bookings/lookup` accepts a booking reference and email, rate-limited to 20 requests per 15 minutes, and returns the full booking read-only.
"I want to know how much I still owe and when it's due."	`GET /api/bookings/:id/payment` returns totalPackagePrice, amountPaid, remainingBalance, and paymentDueDate, computed server-side so the client never calculates the balance itself.
"I don't want staff changing my booking's price by accident through a bug."	`PATCH /api/bookings/:id` whitelists only destination, dates, traveller counts, and notes; status and price fields are rejected even if present in the request body.
"I want my passport number to be safe."	The field is encrypted with AES-256-GCM before it is written to the database, and masked in the client UI until explicitly revealed.
"I want to open my e-ticket without my login being exposed in a link I might forward."	`GET /api/documents/:id/link` issues a single-use, 90-second signed token scoped to one document; it is rejected if used as a general API credential.

User requirement	System requirement
"As a booking agent, I want to see which bookings are mine to handle."	`PATCH /api/bookings/:id/assign` sets assignedStaffId; `GET /api/bookings` returns the assignee on every booking so the client can filter and display it.
"As an administrator, I don't want a deactivated staff member to keep working until their token expires."	Every authenticated request re-checks the staff member's status in the database, not just the token's validity, so deactivation takes effect immediately.

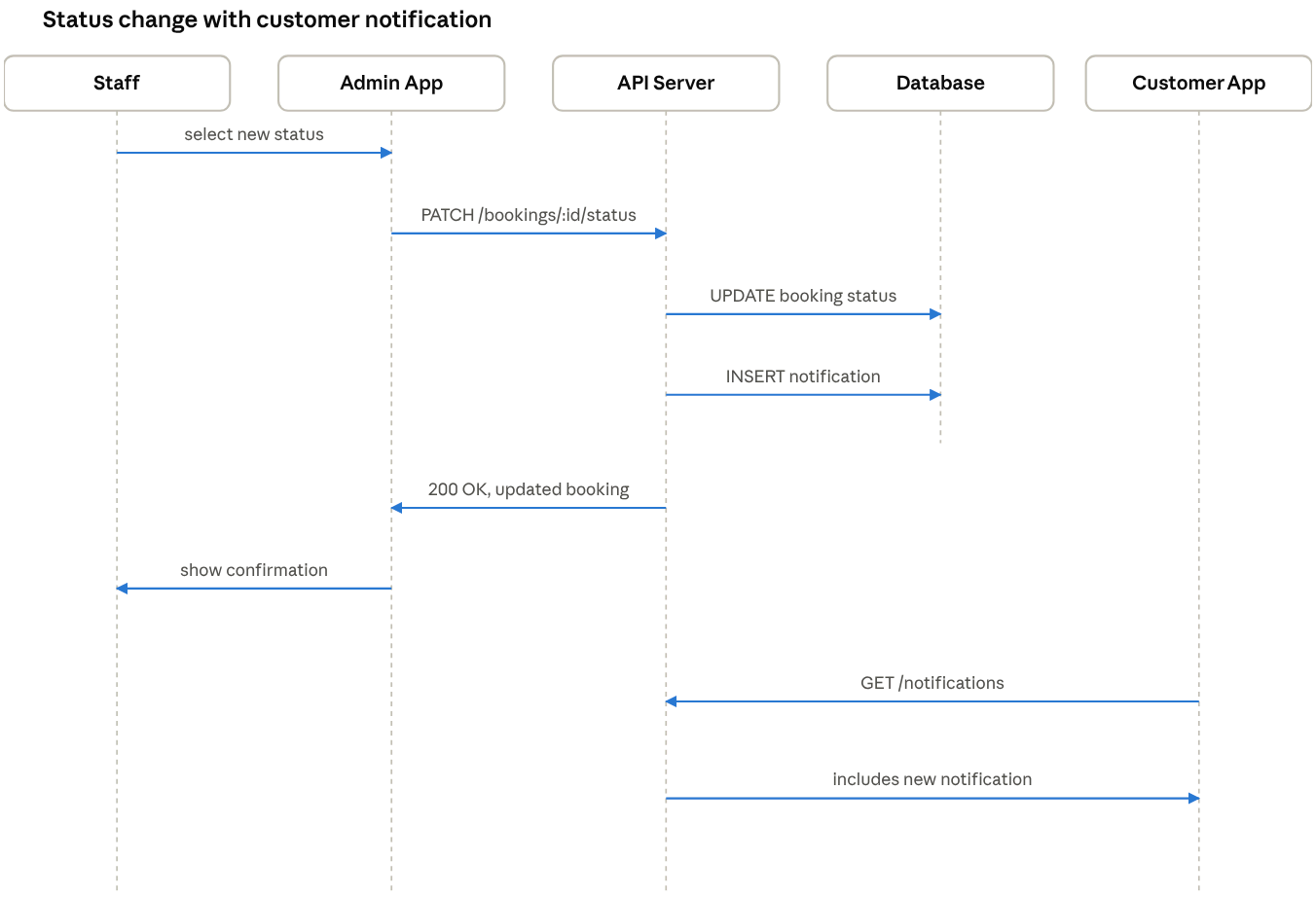


Class diagram: the Flutter client's domain model

This is the client's domain model, not a copy of the database schema (that is the ER diagram above): `Booking` composes `FlightInfo`, `HotelInfo`, and `TransferInfo` as value objects parsed straight from one API response, and exposes computed properties such as `daysUntilTravel` and `isCancelled` that exist only in memory, never in the database. These classes are implemented in `flutter_app/lib/models/`.

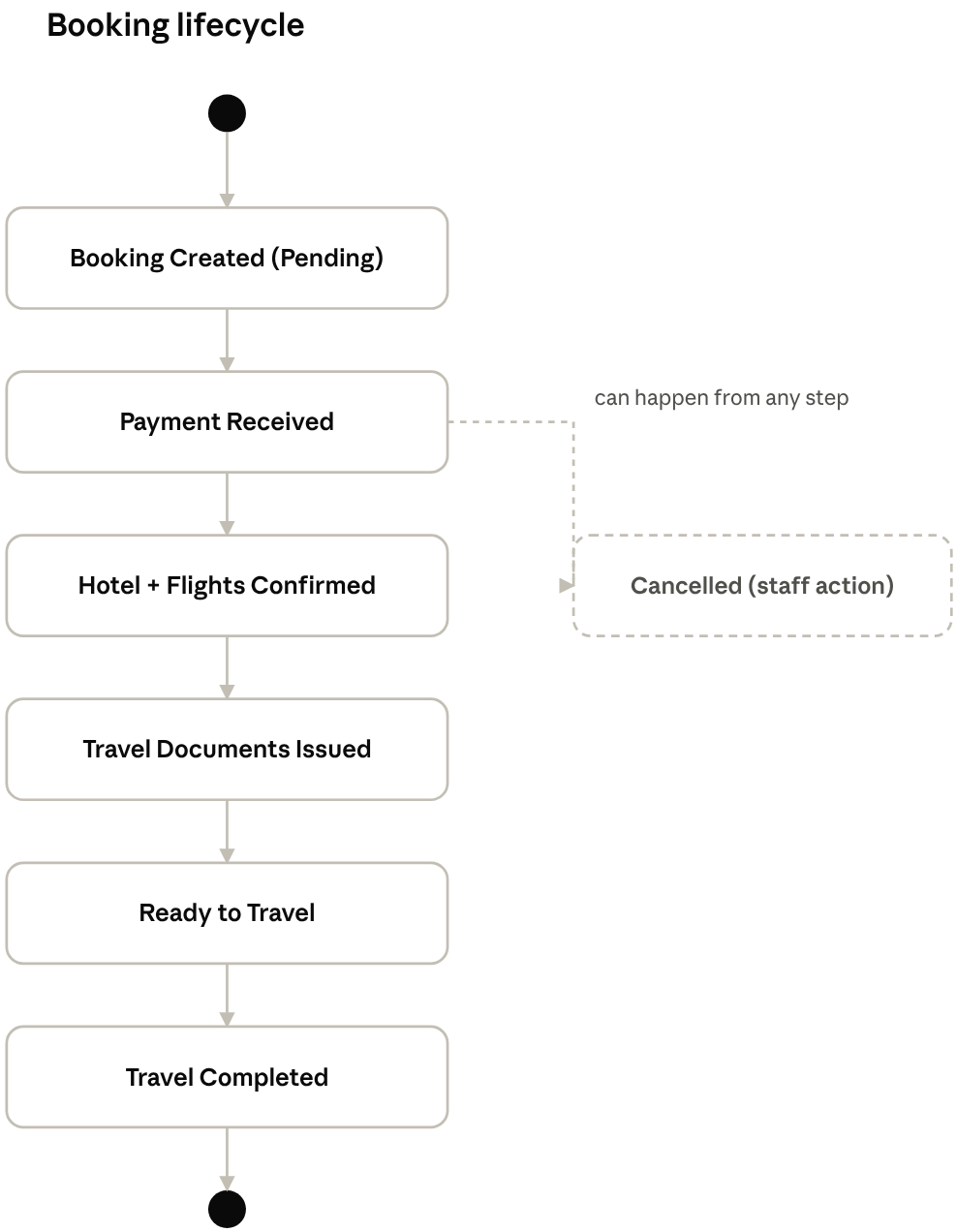


Sequence diagram: public booking lookup



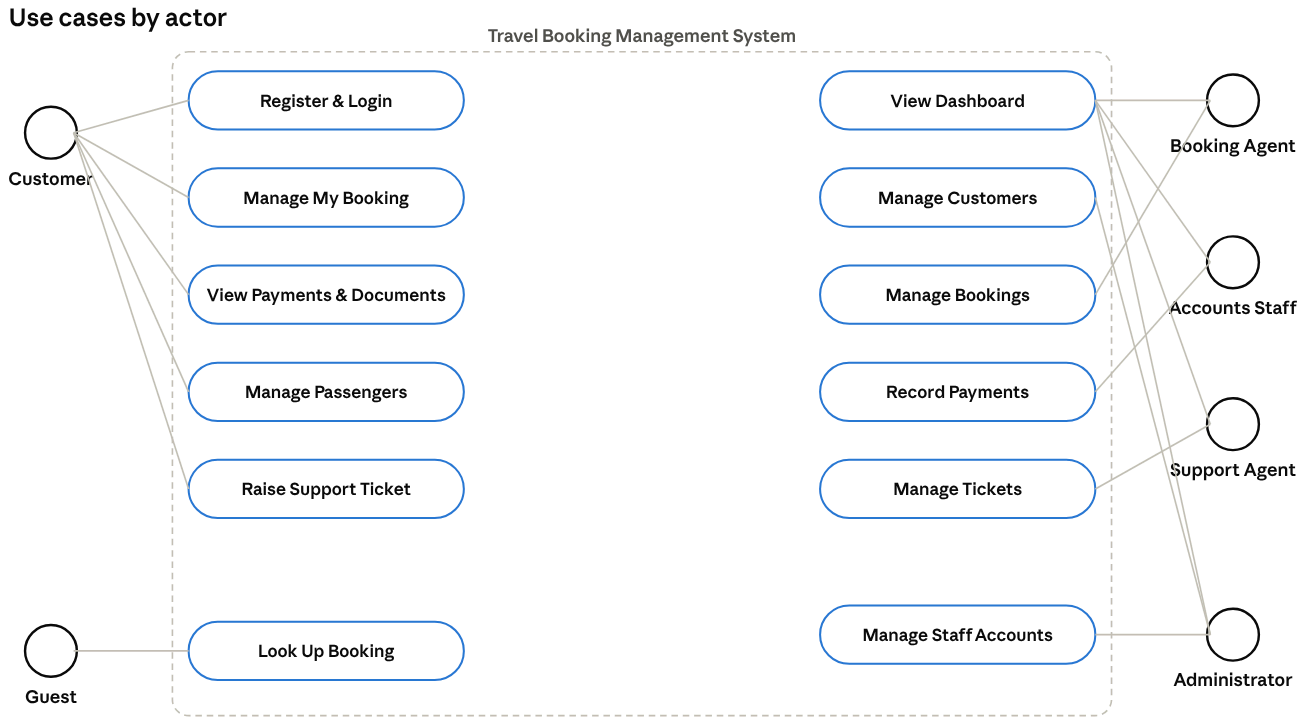
Sequence diagram: status change with notification

The status change example shows why decoupling the booking update from the notification matters: the customer does not need to be online when staff act, because the notification is written to the database and simply appears the next time their app polls for it.



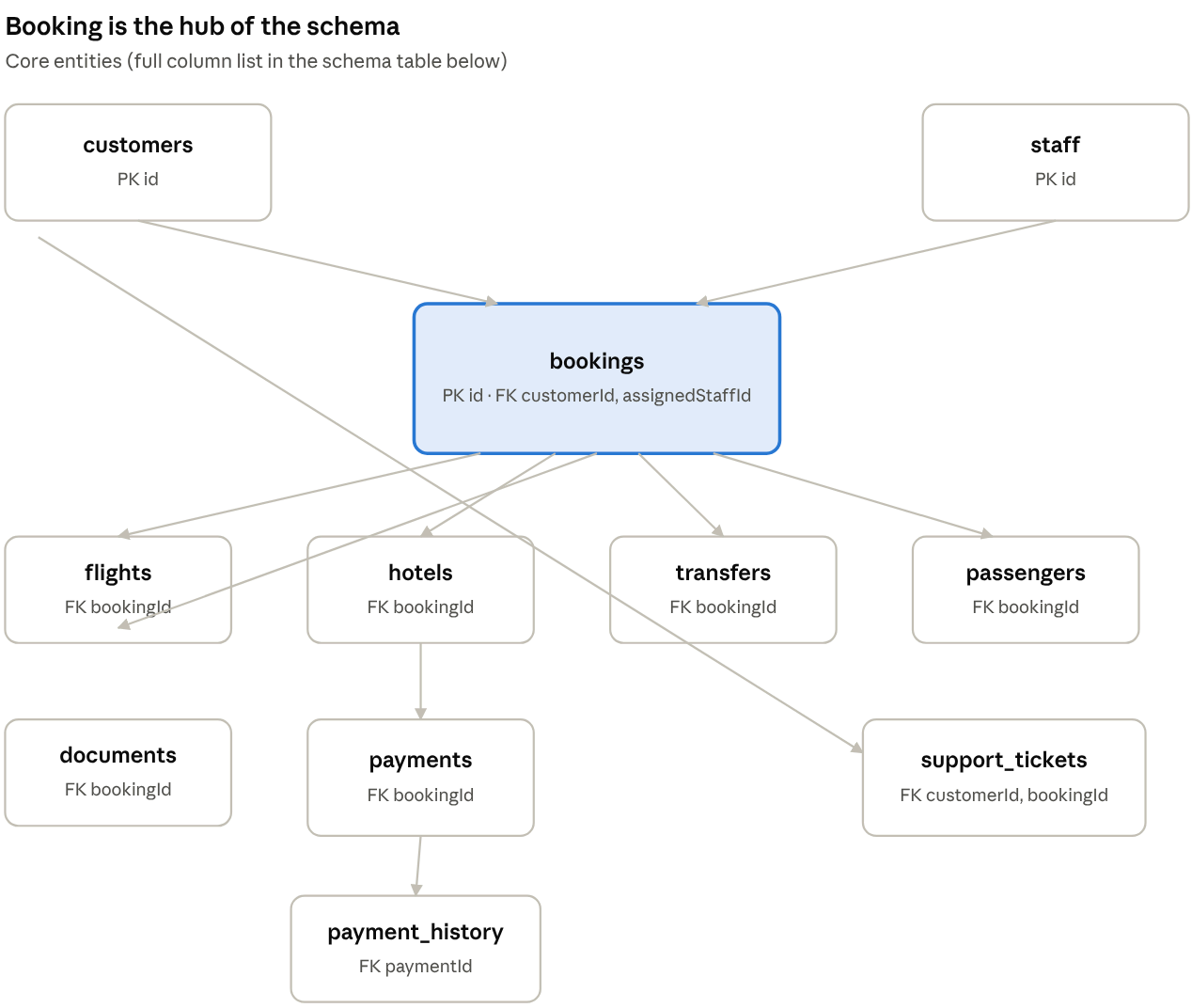
Activity diagram: booking lifecycle

Each arrow corresponds to a staff action on `PATCH /api/bookings/:id/status`, which triggers a customer notification. Cancellation is reachable from any state, matching the specification's status list, which places Cancelled and Closed outside the main forward timeline.



Use case diagram: customer, guest, and staff roles

For readability, each use case shows a representative association rather than every actor it touches — for example, every staff role can view the dashboard, and the Administrator can perform every use case on the right. The complete, precise permission matrix is in section 1.3 (Actors) and section 2 (Functional Requirements) above.



ER diagram: bookings as the central entity

Each booking belongs to one customer and may be assigned to one staff member. Flight, hotel, and transfer are each a one-to-one extension of a booking (kept in separate tables rather than columns on bookings, since not every booking has all three). Passengers, documents, and the payment history are one-to-many. The remaining tables — `ticket_replies`, `notifications`, and `contact_notes` — follow the same one-to-many pattern from their parent and are detailed in the schema table below.

10. Database Schema

The schema is in Third Normal Form: every non-key attribute depends on the whole primary key and nothing but the key. Repeating groups (passengers, payment history, flight/hotel/transfer for one booking) are each split into their own table rather than stored

as columns or JSON blobs, which is why Booking has no flight or hotel columns of its own.

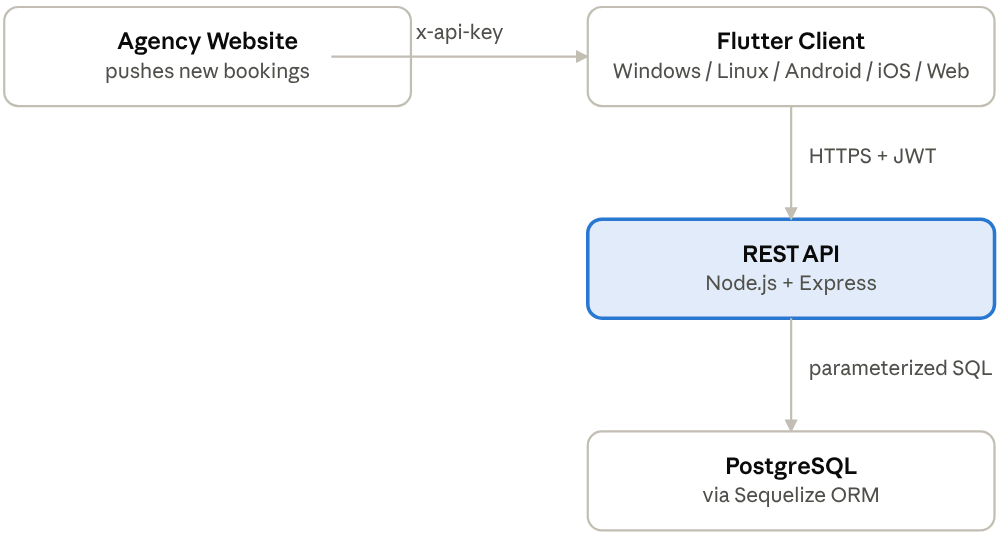
Table	Primary Key	Foreign Keys	Notes
customers	id	—	email unique; passwordHash never returned by the API
staff	id	—	email unique; role and status enums
bookings	id	customerId → customers.id	bookingReference unique; status enum drives the lifecycle
flights	id	bookingId → bookings.id	one-to-one with bookings
hotels	id	bookingId → bookings.id	one-to-one with bookings
transfers	id	bookingId → bookings.id	one-to-one with bookings
passengers	id	bookingId → bookings.id	one-to-many; passportNumber stored AES-256-GCM encrypted
payments	id	bookingId → bookings.id	one-to-one; running totals (amountPaid, paymentStatus)
payment_history	id	paymentId → payments.id	one-to-many; append-only ledger of every payment and refund
documents	id	bookingId → bookings.id	one-to-many; stored filename is random, original filename kept separately
support_tickets	id	customerId → customers.id, bookingId → bookings.id (nullable), assignedStaffId → staff.id (nullable)	bookingId is null for General Support tickets
ticket_replies	id	ticketId → support_tickets.id	one-to-many; senderType distinguishes customer from staff replies

Table	Primary Key	Foreign Keys	Notes
notifications	id	customerId → customers.id	one-to-many
contact_notes	id	customerId → customers.id, bookingId → bookings.id (nullable), staffId → staff.id (nullable)	staff contact log, independent of tickets

Normalization notes:

- **1NF**: every column holds a single value — passenger details are rows in their own table, not a delimited string on the booking.
- **2NF**: every table uses a single-column surrogate key (id), so partial-key dependency cannot arise.
- **3NF**: derived values are not stored redundantly where they would need separate maintenance — for example, a booking's remaining balance is computed from totalPackagePrice and amountPaid at read time rather than stored as a column that could drift out of sync.

Client apps call a REST API backed by PostgreSQL



One codebase, five platforms. One API, two client roles (customer and staff), each with its own token.

3-tier architecture: client, API, database

The Flutter client never talks to the database directly; every read and write goes through the REST API, which is the single point where authentication, authorization, and validation are enforced.

12. Security Measures

Threat	Mitigation
Password theft	bcrypt hashing, 10 salt rounds; never returned in any API response
Token replay after logout	Deactivated staff tokens are rejected immediately by re-checking account status on every request, not only at token expiry
Mass assignment (overwriting status, price, or ownership via an edit request)	Field whitelisting on every update endpoint; a request can only set the fields the endpoint explicitly allows
Brute-force login attempts	Rate limiting: 100 requests / 15 minutes on auth endpoints
Booking reference enumeration	Rate limiting: 20 requests / 15 minutes on the public lookup endpoint; references are generated with cryptographically random characters, not sequential
Passport data exposure	AES-256-GCM encryption at rest; masked in the client UI by default; never included in list views, only the single-passenger detail view
Reset token interception	Reset tokens are stored as a SHA-256 hash, not in plain text; a stolen database dump cannot be used to reset accounts
Unauthorized website integration	The booking-sync endpoint requires a shared API key compared with a timing-safe equality check, resisting timing attacks
Document link leakage	Download links expire after 90 seconds and are scoped to one document; they are rejected if presented as a general login token

Threat	Mitigation
SQL injection	All database access goes through Sequelize's parameterized queries; no raw SQL string concatenation
Cross-role privilege escalation	Every write endpoint checks the caller's role server-side (e.g. only Administrator and Accounts Staff can record a payment), independent of what the client UI shows

13. Testing

13.1 Approach

Backend testing is end-to-end: a real PostgreSQL database and a running Express server are exercised through actual HTTP requests, not mocks, so a pass genuinely means the API behaves correctly over the wire. Flutter testing is unit-level: the models that parse real API responses, formatters, validators, and business-logic helpers (the booking timeline, role permission checks) are tested against response shapes captured from the live backend.

13.2 Results

Suite	Count	Result
Backend end-to-end (`backend/scripts/smoke-test.js`)	62 tests	62 passed, 0 failed
Flutter unit tests (`flutter_app/test/`)	43 tests	43 passed, 0 failed
`flutter analyze`	—	0 issues

13.3 Backend coverage

- Authentication: registration, login, staff login, forgot/reset password, token rejection after logout
- Authorization: every role boundary (e.g. Accounts Staff cannot edit a booking; Support Agent cannot record a payment)

- Mass-assignment protection: confirms status and customerId cannot be set through the booking edit endpoint
- Data protection: confirms passport numbers are encrypted in the database and excluded from list responses
- Business rules: traveller-count limits on passengers, overpayment rejection, return-date-before-travel-date rejection
- Document security: signed link issuance, expiry, and that a link token cannot be reused as a login token
- Rate limiting on authentication and public lookup endpoints

13.4 Client coverage

- Model parsing against real captured API response shapes (booking, payment, ticket, notification, staff, admin dashboard)
- Derived logic: remaining balance calculation, booking timeline progress, overdue payment detection
- Formatters: currency, dates, date ranges, status labels, masked secrets
- Role permission helpers: confirms each staff role sees only the actions the server would actually allow it to perform

13.5 Manual testing

Every screen was exercised manually against the running backend using seeded demo data (`backend/scripts/create-demo-booking.js` and `fill-demo-booking.js`), covering the full customer journey from registration through to viewing a confirmed, fully-populated booking, and the full staff journey across all four roles.

14. Known Limitations and Future Work

- Session tokens are stored with `shared_preferences` rather than platform secure storage (Keychain / Keystore), a deliberate trade-off to keep every target platform (including desktop) building reliably within the project timeline. The server still enforces expiry and immediate revocation on logout.
- The admin portal does not yet have its own automated end-to-end UI tests (only unit tests on its models and permission logic); the backend endpoints it calls are fully covered by the 62 backend tests.

- Passenger management for a booking is available to the customer and to any authenticated staff member via the same endpoint, but there is no dedicated staff screen for it, matching the specification, which lists Passenger Management only under Customer Features.
- Future enhancements could include push notifications (the in-app notification system already exists and could be extended), multi-currency pricing, and a customer-facing cancellation workflow beyond the existing support-ticket category for it.